VASTRANGAM

VASTRANGAM GROUP ERP — PROJECT REPORT

Every module, what is actually built, and the order to build the rest.

Vastrangam · Ethnic Fashion (Go4Fashion) · Adini Couture — Surat Report date: 12 August 2026

HOW TO READ THIS

Three claims are made about every module, and they are kept separate on purpose:

	Meaning
Proven	Code exists, it has been run on your own files, and it reproduces figures from your own reports. The test that proves it is named.
Built	Code exists and runs. It has not been checked against your real numbers.
Specified	A document describes it. No code.

Nothing here is called done because a document describes it well.

Section references like §A4 or §7 point into `Vastrangam_ERP_ALL40Apps_and_Modules1.pdf`, which is Book A and carries all three books plus the accounting deep-dive and the company data prompt. Where a rule comes from a different file, that file is named.

PART 1 · WHERE THE PROJECT ACTUALLY STANDS

There are three bodies of work in this repository. None of them talks to the other two.

1.1 The eighteen single-file tools — `brand/suite/deep/`

AskPrint · B2B Credit · CRM & Customer 360 · D2C Sales · CEO Dashboard · Documents & eSign · Export · Group Consolidation · Helpdesk & Live Chat · Module 01 Unified · Module 02 Unified · Marketplace OMS · Order Manager · POS · Procurement · Quotes & Proforma · Report Builder · Vendors.

Thirty-six built HTML files (each tool in an ERP and a Vastrangam variant). Every one is a browser tool storing its data in its own `localStorage`.

Status: Built, not proven. And they are, precisely, the problem the master spec was written to solve. §A8 records the gap in its own words:

> *"15+ separate HTML files, separate storage — no cross-module flow."*

A sale entered in the D2C tool never reaches the Dashboard tool. That is not a bug in any one of them; it is what eighteen separate storages means.

The same spec, §10, is equally clear about their value:

> *"They are the reference implementation each Track-B module must match."*

So: the logic is the asset, the isolation is the defect. Every rule proven in these tools carries forward. None of the eighteen storages does.

1.2 The AI Studio — `app/` and `brand/suite/aiengine/`

Thirty-eight modules behind an Express server with real file storage: AI Content Engine (the 14-section product report, humanized), Image Studio (layers, presets, transform, background removal, circle export), Design Studio, Video Studio (real MP4 through ffmpeg, 1080×1920, which no browser can do alone).

Status: Built. It runs, it produces work you can use today, and it is the one piece already delivering value with no dependencies on anything else. It stores its work in one JSON document rather than a database.

1.3 The rules engine — `engine/`

Python. Staff pay, karigar production, cost allocation, performance, and thirteen validation gates that block a build which does not tie out.

Status: Proven. Against your own files:

Figure	Engine	Your file
FY2025-26 payroll	₹9,75,648.80	₹9,75,649
FY2025-26 paid	₹10,09,023.00	₹10,09,023
Logged hours · designs	10,388 · 159	10,388 · 159
Karigar earned	₹34,27,498.25	₹34,27,498.25
Karigar paid · outstanding	₹29,12,868 · ₹5,14,630.25	same
Pieces · complete sets	54,436.5 · 30,811	same
Blended hourly, all ten staff	164.43 · 63.49 · 39.13 · 82.14 · 160.71 · 117.86 · 53.57 · 53.57 · 36.96 · 34.78	same

195 checks pass. The karigar figures are recomputed from 1,695 transaction rows, not read off a totals row, and all 158 designs match their recorded complete-set count.

1.4 What this means

One module of twenty is proven. One is built and useful. Eighteen tools hold correct logic in the wrong architecture. Everything else is a document.

That is not a bad position. It means the expensive part — working out what the rules actually are, on real data, against real disagreements — is done for the hardest module, and the pattern for doing it is established.

PART 2 · THE CANONICAL MODULE LIST

The source documents enumerate the modules three times and the three do not agree: §A2 lists 16 domains, §5 lists 10 domains and about 35 apps, §6 lists 20 core modules. The competitive gap analysis then adds 12 more features.

This is the single list. Twenty modules, the §6 numbering, with the §A2 domains and the 12 additions folded into their homes.

#	Module	What it does	Today	Phase
1	**Identity & Access**	Auth, RBAC, company switching, sessions, per-company scoping	Specified	1
2	**Master Data**	Brands, designs, items/SKUs, colours, sizes, categories, HSN, tax rates, vendors, customers, units, locations. *+ Master-Data Hygiene (dedup/merge)*	Specified	1
3	**Procurement**	RFQ, PO, GRN, vendor portal, 3-way matching, vendor priority P1→P2→P3, scorecards	Built (Procurement, Vendors tools)	3
4	**Inventory**	One stock number per SKU × location × stage, transfers, adjustments, batches, dead-stock. *+ Kit/Combo SKU, + Barcode Operations, + WMS bins*	Built (lite)	3
5	**Manufacturing**	Production orders, 10-stage pipeline, BOM, job work, sample approval, QC, pooled set-completion, piece-rate costing	**Proven** (karigar engine)	3
6	**HR & Payroll**	Attendance, effective-dated salary, karigar earnings, advances, leave & festival rule, payroll, slips, appraisal	**Proven** (staff engine)	2
7	**Sales — D2C**	Shopify sync, cart, checkout, partial COD, loyalty, customisation orders. *+ Subscriptions*	Built	4
8	**Sales — Marketplace**	Amazon/Flipkart/Myntra/Meesho/Ajio/Nykaa/JioMart order pull, unified queue, settlement reconciliation. *+ Listing & Catalog Manager, + Repricing Engine*	Built (OMS tool)	4
9	**Sales — B2B**	Proforma → order → invoice, credit limits, tiers, ageing, IndiaMART leads	Built	4
10	**Sales — Export**	Commercial Invoice + Packing List, LUT bond, FIRA, IGST-refund tracking	Built	4
11	**Returns**	Courier / customer / wrong-return, lost-inventory write-off, dead stock. *+ NDR/RTO workflow*	Specified	4
12	**Finance — Books**	Double-entry, GST (CGST/SGST/IGST), TDS, TCS, ITC & GSTR-2A/2B matching, bank recon, period lock, audit trail	Specified	5
13	**Finance — Reports**	P&L per company + group, Balance Sheet, GSTR-1/3B/9, ageing, cash flow, ratios, budget vs actual	Built (Reports, Group Consolidation)	5
14	**Marketing**	Content calendar, campaigns, ROAS, influencer CRM, asset library. *+ Events, + Forms & NPS*	Specified	6
15	**CRM**	Unified customer 360 across D2C + B2B + export + walk-in, lifecycle (VIP at 7, win-back at 90d), loyalty	Built	4
16	**AI Command Centre**	Content Engine, Image Studio, Design Studio, Video Studio, model routing, 8 AI modules	**Built and in use**	6

#	Module	What it does	Today	Phase
17	**Communications**	WhatsApp IN/OUT/REPORT/LEAVE/ADVANCE + broadcast, email, SMS, notifications	Specified	2 & 6
18	**Documents**	Auto-PDFs, eSign, Drive sync, archival. *+ Knowledge Base / SOP wiki*	Built (Documents & eSign)	6
19	**Dashboards**	Role-specific — Admin, Manager, Staff, Karigar, Customer	Built (CEO Dashboard)	6
20	**Settings**	Provider config, env, tax rates, voucher series, roles, integration health. *+ Automation/Workflow engine*	Specified	1

Fleet (own delivery vehicles) and Recruitment (ATS) are listed in the gap analysis as optional. They are not in the twenty and should not be until something needs them.

PART 3 · THE ONE LAW, AND HOW CODE ENFORCES IT

§A0 states the whole design in one sentence:

> **A business event is entered once and flows everywhere it belongs. No module re-enters data another module already has. No number is maintained separately from its source.**

3.1 The seven shared entities

One set of master entities every module reads and writes. This is the physical reason a sale can touch stock and books in the same transaction (§A3):

Company (every row carries `company_id`) · **Item/SKU** · **Party** (customer, vendor, karigar, staff — one identity) · **Stock** (item × location × stage) · **Ledger/Voucher** (the single financial truth) · **Order** · **Event bus**.

3.2 The fourteen cascades

§A4 enumerates every required flow. The ones that matter most, and what breaks without them:

1. **CRM → Sales** — a won lead becomes an order carrying customer, price list, credit tier.
2. **Sales → Inventory** — reserve or deduct the single stock number; auto-delist at zero across every channel.
3. **Sales → Accounting** — Dr Debtor/Bank, Cr Sales + Output GST, with place-of-supply deciding IGST versus CGST+SGST.
4. **OMS → Sales** — each marketplace order normalises into a Sales Order, idempotent by external id.
5. **OMS → Settlement → Dispute → Accounting** — per-line reconciliation; every variance opens a categorised claim with an evidence pack.

6. **Sales/OMS → Logistics** — one-click label, AWB back onto the order, COD remittance reconciles the second leg.
7. **Purchase → Inventory + Accounting** — GRN adds stock, 3-way match gates the payable, GST becomes ITC.
8. **Manufacturing → Inventory + HR + Accounting** — production adds finished stock, computes pooled piece-rate earnings, posts karigar wages, rolls cost-per-piece into design P&L.
9. **HR → Accounting** — payroll on effective-dated salary posts Salaries.
10. **Accounting → BI** — *every dashboard number is a query on the ledger, never a separate counter.*
11. **Returns → Accounting + Inventory + CRM** — credit note and refund; a wrong-return is a dead-stock loss and is **not** restocked.
12. **AI Studio → Marketing → Sales/OMS** — generated assets attach to the SKU, become listings, get published, ROAS tracks back.
13. **Automation** — any event triggers any action across modules.
14. **Notifications** — WhatsApp, email and SMS fire from any module's events.

Cascade 10 is the one that decides whether the system is trustworthy. The moment a dashboard keeps its own running total, the numbers start to disagree with the books and nobody can say which is right.

3.3 The gates that enforce it

The engine already runs thirteen. They are the model for the rest, and they exist because each one caught something real:

Gate	Why it exists
Every log resolves exactly one row per employed staff-month	Zero matches is an **error**, not zero — that is how someone earns ₹0 unnoticed
Every category resolves to an Hours Reference row	A missing row would read as a month of zero hours
Flat staff earn exactly their salary, whatever the attendance	A raise may change it; attendance never may
Piece-rate staff never draw on Staff Master	Their rate lives in the production file, by definition
Reconciliation FY earning = the sum of the monthly rows	No figure maintained twice
Every roster mismatch is listed	Inactive-but-working is a flag, never a silent include
Design cost + unallocated labour = total payroll	Unallocated labour is real and hiding it flatters cost-per-piece
Karigar earnings = the sum of parsed source rows	Nothing invented, nothing dropped
Combined = the sum of each period, per unit	The Power BI rule: consolidated is `=SUM(above)`, never a separate query
Every production row's qty × rate = its value	The cheapest arithmetic in the file, and the most worth checking
No design reports more sets than its scarcest required piece	Caught a real error — see Part 6
Every source row is matched or in Needs Review	Nothing is ever dropped
No logic references a person by name	The gate that keeps all the others true

A build that fails a gate does not ship. `run.py` exits non-zero, so a broken build cannot be mistaken for a working one.

PART 4 · THE BUILD ORDER — MODULE 1 TO LAST

Eight phases. The Definition of Done for each is the spec's own (§C.4) — those are already good tests and there is no reason to invent new ones.

The rule that matters more than the order: Phase N+1 does not start until Phase N's tests pass.

Phase 0 · The core (before any module)

The thing that does not exist yet and without which everything else repeats the current mistake.

- One database. Every business table carries `id`, `company_id`, `created_at/by`, `updated_at/by`, `deleted_at`, `version`.
- The seven shared entities from §3.1.
- An event bus — modules announce, they do not call each other directly.
- The service interfaces from §3.3 of the spec: `DatabaseService`, `WhatsAppService`, `AIService`, `PaymentService`, `ShippingService`, `AutomationService`, `StorageService`. **No provider SDK ever appears in business logic.**
- An audit trail that cannot be switched off (Part 8.4).
- Soft delete everywhere. Staff get "deactivate", never "delete".

Done when: a row written by one module is read by another, and every write is audited.

Phase 1 · Foundation — Modules 1, 2, 20

Three companies with the correct prefix and brand-code split (Vastrangam VS/VS · Ethnic Fashion EF/**GF** · Adini AC/AC — the second one deliberately differs and confusing it corrupts every report). Roles and company switching. Staff and karigar master. Design library. The SKU model. Vendors, customers.

Done when: an admin can create a design with 5 colours × 7 sizes in under five minutes.

Phase 2 · HR & Karigar — Modules 6, 5 (payroll half), 17 (WhatsApp)

The Python engine lifts in whole. It is already proven; it does not get rewritten, it gets wrapped. Attendance capture, the effective-dated logs, the payroll run, slips, karigar earnings, advances, the festival-leave rule.

Done when: a full month's payroll runs end to end with zero manual touch.

This is the phase where the whole approach pays off or does not. If the proven engine runs unchanged against the new core, the pattern is right.

Phase 3 · Make — Modules 4, 5, 3

Stock by SKU × location × stage. The 10-stage production pipeline. BOM. Sample workflow. QC. Job work. Kit/combo SKUs. Barcode operations.

Done when: three production orders — one self, one full job work, one partial — run to completion.

Phase 4 · Sell — Modules 7, 8, 9, 10, 11, 15

Shopify sync. Marketplace order pull. Settlement reconciliation and the claims engine. B2B with credit limits. Export with CI, PL and LUT. POS. Returns. Customer 360.

Done when: a full week of operations runs with every channel live and settlements reconciled.

The claims engine is the money. Today disputes are manual and about half are lost.

Phase 5 · Money — Modules 12, 13

Double-entry through **one posting engine** — no voucher type gets its own ledger logic. GST. TDS/TCS. ITC and GSTR-2A/2B matching. Bank reconciliation. Period locking. The full report suite.

Done when: one month of books closes cleanly and GSTR-1 + GSTR-3B generate and verify.

Phase 6 · Reach — Modules 14, 16, 17, 18, 19

Marketing, the AI Command Centre (already built — this phase wires it to the catalogue rather than building it), communications, documents, dashboards.

Done when: 50 listings and a month of content generate, and loyalty redemption works.

Phase 7 · Cutover — Module 20 completion

Opening balances at the cutoff. Parallel run. Training. Smoke tests. Go-live.

Done when: the first real transaction posts in the new system.

PART 5 · STACK, AND THE BRIDGE TO SUPABASE

Decision: one local core first, then host it.

The spec's destination is Next.js 15 + Supabase (Postgres 16, Auth, Storage, Realtime, RLS) on Vercel, with n8n and Ollama on a Hostinger VPS — eight phases, thirty-two weeks, a dev team, and paid accounts from week one.

That destination does not change. What changes is that you do not pay for it, or wait for it, before anything works.

How the bridge stays honest

§3.3 of the spec already requires it:

> *"No provider SDK in business logic — always behind a service interface. Swapping Supabase→Firebase, Claude→GPT, Interakt→Wati = a config change, never a rewrite."*

So Phase 0 builds those interfaces first, with local implementations behind them:

Interface	Local now	Hosted later
`DatabaseService`	Postgres or SQLite on your machine	Supabase Postgres 16 + RLS
`StorageService`	local `data/` folder	Supabase Storage or Drive
`AIService`	already built, key in `.env`	unchanged
`WhatsAppService`	stub that logs	Interakt
`PaymentService`	stub	Razorpay / PayPal
`ShippingService`	stub	Shiprocket
`AutomationService`	in-process rules	n8n

Schema identical from day one, including `company_id` on every table. Row-level security is enforced in the middleware locally and by Postgres RLS when hosted — the spec calls for both anyway, as defence in depth.

The migration is then a connection string and three real credentials, not a rewrite.

What this costs you

Honest accounting of the trade:

- **You lose:** multi-device access until you host, real WhatsApp until Interakt is paid for, live marketplace pulls until those APIs are approved.
- **You gain:** a working system in weeks instead of a 32-week wait; every rule proven on your real files before a single rupee of hosting; and the freedom to stop, change direction, or hand it to a different builder without having bought anything.

PART 6 · WHAT IS PROVEN, AND WHAT IS NOT

6.1 Proven — and what proving it found

Working through the real files did not just confirm the rules. It corrected them.

The November threshold change. FY2025-26 payroll came out ₹8,006 short. The cause: the threshold moved from 280 to 270 **hours** in November 2025, and the **days** threshold moves with it, 28 to 27 — 270 hours at a ten-hour day. Holding days at 28 while hours dropped understates the year. Your own `FY202526_Staff_Productivity_Report.xlsx` has a Threshold Log that says exactly this; it was in the file the whole time.

The set-completion rule was wrong. It took the smallest *populated* slot, which errs in both directions at once. Design V518 — 22 tops, 22 dupattas, **no bottoms** — read as 22 finished sets when none can ship.

GreenKurtiPlazzo — 854 tops, 855 bottoms, 194 dupattas — read as 194 when 854 two-piece sets are done. The minimum belongs over the pieces the set *requires*.

And composition cannot be read from the name. An *Anarkali Plazo Set* contains a dupatta. A *Kurti Plazo Set* does not. Neither name says so. Each entry in `engine/fixtures/set_types.json` is the only one of six possible combinations that reproduces the recorded count for every design of that type — 41 designs constrain the first, 34 the third. With that table read as data, all 158 designs match.

The stray header. Your uncorrected staff workbook carries a wrong header on row 1 — four names belonging to people not employed that year, sitting above twelve correct block headers. A parser trusting row 1 hands four people's entire year to four others, and nothing downstream would say so. The structural rule — *a header is real only when a date sits beneath it* — catches it. Both files read the same 1,435 marks for the same ten people. It also explains why the earlier productivity report lists those four in FY2025-26.

The three-state month. Not employed, no data, and genuinely absent are three different things. In FY2025-26 the source held 2,215 blank cells against 152 marked absent. Reading blanks as absences would have scored eight people as failing months they never worked.

6.2 Not proven

Modules 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20 have never been run against your real data. The eighteen tools produce output; nobody has checked it against your own books.

Nothing in the settlement, GST, or accounting path has been tested at all. That is the largest untested surface and it is also where the money is.

PART 7 · THE DATA MODEL

PostgreSQL. Every business table carries the audit columns. Table groups (§7):

Foundation — companies, users, user_companies, audit_log, integration_errors, settings_environment.

Master data — brands, designs, design_categories, colors, sizes, items, item_aliases, hsn_codes, gst_rates, vendors, vendor_materials, customers, customer_addresses, countries, states, currencies, fx_rates, locations.

Inventory — stock (PK item × location × stage), stock_movements, batches, stock_adjustments, opening_stock.

Procurement — purchase_requisitions, purchase_orders, purchase_order_items, grn, grn_items, vendor_invoices, three_way_match.

Manufacturing — production_orders, production_stages, bom, bom_items, samples, karigar_assignments, karigar_reports, qc_records, performance_flags.

Sales — sales_orders, sales_order_items, invoices, invoice_items, marketplace_orders_raw, marketplace_settlements, marketplace_settlement_lines, b2b_orders, b2b_credit_ledger, export_orders, customization_orders, returns.

Finance — chart_of_accounts, voucher_series, journal_entries, journal_lines, gst_returns, gst_input_credit, tds_entries, tcs_entries, bank_accounts, bank_transactions.

HR — staff_salary_history (effective-dated — the payroll source of truth), attendance, eod_reports, leave_requests, advance_requests, payroll_runs, payroll_slips, karigar_earnings_summary, piece_rates, task_threshold_rates.

7.1 The SKU rule

```
BRAND → DESIGN → STYLE-VARIANT → SKU
SKU string = {BRAND}-{DESIGN}-{COLOR}-{SIZE}      e.g. VS-MUSPUR-LAV-M
```

Analytics always query the structured fields. Never substring-match the SKU string. The string is for humans.

7.2 Effective dating is not optional

`staff_salary_history` is named in the schema as the payroll source of truth, and the pattern generalises. Anything that changes over time — salary, threshold, tax rate, piece rate, pay basis, price — is a log, never a column:

```
set_value(key, from_date, value):
    close the open row     ->  to = from_date - 1 day
    append the new row     ->  (key, from_date, NULL, value)
    never touch rows that already ended
```

Resolving a value for a month returns **exactly one row**. Zero matches is an error, not zero. A future-dated row starts working by itself when that month arrives.

`engine/vastrangam/logs.py` implements this and is tested. GST rates must use it too — the Busy prompt §4 is explicit that *"tax rate must be versioned by effective date, not a single static field, so historical invoices remain correct."*

PART 8 · MONEY, COMPLIANCE AND MIGRATION

8.1 The 10 Golden Rules

From the Power BI master. They govern every report the system produces:

1. **Zero missing entries** — every row from every sheet, no sampling, no row limits, ever.
2. **No hardcoded values** — every number is a live formula against its source table.

3. **INR only.**
4. **Auto-detect the financial year** from the data; label everything dynamically.
5. **No overlapping elements** — charts, KPI boxes and tables never cover each other.
6. **No excessive blank rows or columns** — every row earns its place.
7. **Graphics mandatory** — minimum one chart and one formatted table per sheet.
8. **All output sheets present** and validated before saving.
9. **Dynamic data tables** — processed data goes to named staging tables so ranges expand.
10. **Dynamic formulas** — KPI boxes reference table columns, never fixed cells.

8.2 The reporting structure — every table, no exceptions

```
| Company        | Metric 1   | Metric 2   |
| Vastrangam     | =SUMIFS... | =SUMIFS... |
| Ethnic Fashion | =SUMIFS... | =SUMIFS... |
| Adini          | =SUMIFS... | =SUMIFS... |
| CONSOLIDATED   | =SUM(above)| =SUM(above)|
```

The consolidated row is `=SUM()` of the three above it and **never a separate query**. Visually distinct — navy background, white bold. The engine gate *"Combined = the sum of each period's columns"* enforces the same discipline in code.

8.3 The financial formula chain

```
NET_PURCHASE      = TOTAL_PURCHASE − PURCHASE_RETURN
GROSS_B2B_NET     = GROSS_B2B_SALES − B2B_RETURN − FREIGHT_TOTAL
GROSS_B2C_SALES   = SELLING_PRICE_B2C − TOTAL_RETURN_B2C
NET_B2C_SALES     = GROSS_B2C_SALES + GROSS_B2B_NET

NET_REVENUE       = NET_B2C_SALES
COGS              = NET_PURCHASE
GROSS_PROFIT      = NET_REVENUE − COGS

TOTAL_EXPENSES    = FREIGHT + EXPENSES + STAFF_PROD_COST + KARIGAR_WAGES + JOGINDER_WAGES
NET_PROFIT        = GROSS_PROFIT − TOTAL_EXPENSES
```

Where the marketplace selling price is:

```
Selling_Price_Calc = Price − Commission − Fixed_Fee − Shipping − GST_18% − TCS − TDS
```

And returns are costed by type: **customer** ₹20/pc · **courier** ₹5/pc · **wrong return** the full selling price, written off as dead stock and **not returned to inventory**.

`STAFF_PROD_COST` and `KARIGAR_WAGES` are the two figures the engine already produces.

8.4 Compliance that is not negotiable

The audit trail is a legal requirement, not a feature. Indian company law (MCA rule, FY2023-24 onward) requires accounting software to log every edit to every transaction — who, what, old value, new value, when — that this logging **cannot be disabled**, and that it is preserved for eight years.

> *Architecturally impossible to turn off. Not a settings toggle defaulting to on.*

Also mandatory: period locking after filing, with the unlock itself logged · year-end closing that carries balance-sheet accounts forward and resets P&L · round-off to a dedicated ledger account, never absorbed into the sale amount (which would corrupt GST) · bill-wise payment allocation, because an invoice can be part-settled across payments · post-dated cheques posting on realisation, not on cheque date.

8.5 The BUSY migration

Proven extractable: 827 masters, 7,352 vouchers, 6,906 billing detail rows. `Tran1.PartyCode1/2` joins `Master1.Code`, so every voucher resolves to a real party.

Do not copy BUSY's schema. It stores every entity in one polymorphic table (`Master1`) and every voucher in another (`Tran1`), with generic fields whose meaning depends on a type code. Efficient for a twenty-year-old desktop product, unreadable and error-prone for anything new. Carry the *concepts* — TDS/TCS as first-class entities, GST per line rather than per voucher, serial numbers, POS as a distinct path into the same ledger, deleted-voucher audit, multiple numbering series.

Cutover: opening balances at 1 April 2026 — capital, banks, open receivables and payables, GST/TDS balances, fixed assets, WIP, all masters (~315 customers, ~330 designs expanded to SKUs with `legacy_busy_code` preserved), opening stock at SKU level. Sixty-day parallel run with daily reconciliation. **BUSY is decommissioned only after the first month's GSTR-1 and GSTR-3B are filed from the new system.**

PART 9 · OPEN QUESTIONS FOR THE OWNER

Answers change the numbers. None of these can be settled from the files.

1. **Does an Anarkali Plazo Set include a dupatta?** The data says yes, unanimously across 41 designs. One line in `engine/fixtures/set_types.json` if not.
2. **How many karigar are active?** Book A §2.3 says eight groups — Sajid, Aamir, Mustakim, Sohrab & Team, Rizwan & Tahid, Ekabot & Team, Shubhankar, Rabiyul & Team. You said six paying units, fifteen people. The engine currently follows what you said.
3. **The roster has holes.** §2.1 lists two staff as "name TBD" and seven more as status-to-confirm. §2.3's historical list holds 29 earning units.

4. **Joginder & Ikram's FY2026-27 rate per piece** for the iron job. FY2025-26 is settled at ₹100/hr. Those months currently report *Unresolvable* — visible, not zero.
5. **FY2026-27 is incomplete** — attendance stops 4 August 2026, September to December are empty calendars, the work report is pending.
6. **The 14-sheet master workbook** — opening stock, purchase, purchase return, B2B, B2B return, freight, B2C, B2C return, expenses, production, stitching rate. That is the entire money side and Phase 5 cannot start without it.
7. **Which pay rule is the company's rule?** Days-scaled is what you pay and what reproduces ₹9,75,649. The universal prompt's §4.1 specifies hours-scaled, which gives ₹9,23,269. Both are computed and reported side by side; only the days figure is paid.

PART 10 · HOW TO START

The first week

Day 1 — decide where data lives. One Postgres database, three companies seeded, every table with `company_id`. Nothing else. Prove it by writing one row and reading it from a second process.

Day 2 — the service interfaces. Seven files, each a thin interface with one local implementation. No business logic in any of them. Prove it by swapping the storage implementation for a stub and watching nothing else change.

Day 3 — the audit trail. Every write goes through one function that records who, what, before, after and when. Prove it by trying to write around it and finding you cannot.

Day 4–5 — Module 2, Master Data. Brands, designs, colours, sizes, the SKU model. Prove it by creating a design with 5 colours × 7 sizes in under five minutes.

Then Module 1, Identity, and Phase 1 is done.

The first month

Phase 2. Lift the Python engine in whole behind the new core, add attendance capture, and run a real month's payroll against your real files. **If the proven engine runs unchanged, the architecture is right.** If it needs rewriting to fit, the core is wrong and it is far cheaper to find that out in month one than in month eight.

The rule to keep

Every module ships with the test that proves it, run against your own files, reproducing a figure from your own records. A module that cannot do that is not finished, however complete it looks.

That is the only thing separating the one module that works from the eighteen that compile.

APPENDIX · RUNNING WHAT EXISTS TODAY

```
# The rules engine — every rule, one line per check
python3 engine/tests/selftest.py

# The same, plus your real files
VAS_CORPUS=Staff_Report_FY_202526.xlsx \
VAS_KARIGAR=Karigar_Production_and_Payment_Report_FY202527.xlsx \
VAS_CORPUS_OLD=<the uncorrected staff workbook> \
python3 engine/tests/selftest.py

# A full run — gates, run log, and the diff against last time
python3 engine/run.py --fy 2025-26 \
    --attendance <file> --work <file> --payments <file> --karigar <file>

# The AI Studio
cd app && npm install && npm start        # then http://localhost:3000
```

`engine/README.md` documents every rule, every gate, and how to correct data without touching code.

Vastrangam Group · Desire to Attire · Surat · Confidential — not to be shared outside the company.
